

The background of the slide is a dark blue gradient. It is decorated with numerous light blue hexagons of varying sizes and orientations. Some hexagons are solid outlines, while others are filled with a lighter blue. A network of thin, glowing blue lines connects various points, some of which are small blue dots. The overall effect is a futuristic, technological aesthetic.

INDUSTRY PROJECT

(Web and Mobile Apps)

08-Feb-2026

Technology Stack

Contents

Core SaaS Architecture & Security

- Technology Stack
- Architectural Considerations
- Data Encryption Standards
- Security and Maintenance Guidelines
- Optimized Code and Quality Standards

The Master-Driven Engine

- The "Master-Driven" Architecture
- No Hard-Coding: The "Rule Engine"
- Organizational Hierarchy and Multitenancy
- Audit Logging Strategy

Super Admin & Organization Setup

- The "Super Admin" Master Setup
- Dynamic Role-based Access Control
- UI/UX & White-Labeling
- Operational "Must-Haves" for SaaS

Dynamic RBAC & Data Onboarding

- Dynamic Organization Setup Workflow
- Bulk Operations & Data Validation

Authentication & Support Functionality

- Authentication and Identity Management
- Critical Additions for Support and Functionality
- Access Enforcement

1. Tech Stack

Layer	Technology	Reason for Choice
Mobile App	React Native SQLite	Cross-platform efficiency with robust libraries for Background Geolocation and Camera/Media handling. Mobile side database for offline functionality with data encryption.
Frontend (Web)	NextJS and React	Handles complex, real-time dashboards and dynamic checklist builders with high performance.
Backend (API)	Python Django	Asynchronous architecture ideal for handling simultaneous GPS pings and high-frequency SOS triggers.
Database (Primary)	PostgreSQL with PostGIS	PostGIS is the industry standard for Spatial/Geographic data , crucial for geofencing and "Live Map" logic.
Data Encryption		All sensitive information is secured using AES-256 bit encryption for the mobile local SQLite database (at-rest) and TLS 1.3 for data-in-transit, while the PostgreSQL server utilizes Transparent Data Encryption (TDE) to ensure end-to-end database integrity.
Real-time Sync	Socket.io or MQTT	Essential for live patrolling "Breadcrumbs" and instant SOS dashboard alerts.
Caching/Queue	Redis	Temp storage for active GPS pings to prevent database write-overload during high-volume shifts.
Cloud Storage		Scalable, secure storage for incident photos, videos, and voice recordings.
Infrastructure	Docker + Kubernetes	Ensures the system remains stable and scalable during peak shift-handover periods.
Integration	SMS, WhatsApp	Ensure multi-channel communication and report sending through WhatsApp

2. Architecture Considerations

Logical Architecture Overview

- **Microservices:** The architecture must follow a Microservices pattern to separate the "Reporting" load from the "Real-time Tracking" load.
- **Geolocation Engine:** The backend should utilize a **Geofencing Service** that compares the device's GPS_current coordinates against the GPS_master coordinates defined in your Site Master to unlock/lock checklists.

Security & Performance Optimization

- **JWT (JSON Web Tokens):** For secure, stateless authentication between the mobile app and the server.
- **CDN (Content Delivery Network):** For fast loading of site-map images and historical patrol photos.
- **Data Archiving:** Move older patrol logs (older than 12 months) to a Cold Storage to keep the primary dashboard snappy.

3. Data Encryption Standards

Security must be handled at two distinct layers:

A. Encryption in Transit (Moving Data)

- **Enforce TLS 1.3:** Disable older, vulnerable versions (SSL, TLS 1.0).
- **HSTS (HTTP Strict Transport Security):** Instruct browsers to only communicate with your server via HTTPS.
- **API Security:** Use **mTLS (Mutual TLS)** for server-to-server communication (e.g., between your backend and the SMS Gateway).

B. Encryption at Rest (Stored Data)

- **Transparent Data Encryption (TDE):** Use cloud-native encryption (AWS KMS or Azure Key Vault) to encrypt the entire database disk.
- **Field-Level Encryption:** For highly sensitive fields like `mobile_number`, encrypt the data *before* it hits the database using an application-level key.
- **Secrets Management:** Never hardcode API keys or DB passwords. Use a Secret Manager.

4. Security & Maintenance Guidelines

- **Idempotency:** Ensure that if a Bulk Upload is triggered twice accidentally, it doesn't create duplicate users (use `mobile_number` as a unique constraint).
- **Encryption at Rest (Field Level):** Use **AES-256** for sensitive master data. In the database, the mobile number might look like `7x82b...` but is decrypted by the application key during the OTP process.
- **Environment Variables:** All infrastructure secrets (Database URLs, SMS Gateway Keys) must reside in `.env` files or Secret Managers, never in the repository.

5. Optimized Code & Quality Standards

To ensure the codebase remains maintainable as the SaaS grows, enforce these guidelines:

- **The "Tenancy Filter" Middleware:** Instead of manually adding `WHERE org_id = ?` to every query, use a **Middleware** or **ORM Hook**. This automatically injects the tenant ID into every database transaction based on the user's session.
- **Dry-Run Role Checks:** Implement a central `hasPermission(user, 'PERMISSION_NAME')` utility.
- **Automated Testing Requirements:**
 - **Unit Tests:** Target 80% coverage for business logic.
 - **Integration Tests:** Specifically for the OTP and Hierarchy logic.
- **Static Analysis:** Integrate tools like **SonarQube** or **ESLint** into your CI/CD pipeline to catch "Code Smells" (like deep nesting or high cyclomatic complexity) before they are merged.

6. The "Master-Driven" Architecture

- Every variable that could change across industries or countries must be stored in **Master Tables**, not in code constants or enums.
- **System Constants:** Store things like OTP_EXPIRY_TIME, MAX_LOGIN_ATTEMPTS, and SUPPORT_EMAIL in a global_config table.
- **Dynamic Dropdowns:** Instead of hardcoding "Departments" or "Designations," create a master_lookups table.
 - *Columns:* tenant_id, lookup_type (e.g., 'DEPT'), display_name, system_code.
- **Feature Toggles:** Use a tenant_features table to enable/disable specific modules (e.g., Reports, Mobile Access) per organization without changing code.

7. No Hard-Coding: The "Rules Engine"

- If a certain role needs access to a new report, the Super Admin shouldn't call a developer.
- **Dynamic UI Navigation:** The Sidebar/Menu should be generated by an API call that filters permissions mapped to the user's role_id.
- **Flexible Branding:** The frontend should fetch a GET /theme config on load and apply these values to CSS variables.

Example

```
:root {  
  --primary: ${api.theme.primary_color};  
  --logo-url: url('${api.theme.logo}');  
}
```

8. Organizational Hierarchy & Multitenancy

- **SaaS Isolation:** Use a `tenant_id` on every database table to ensure data leakage never occurs between different organizations.
- **Adjacency List or Closure Tables:** To make the hierarchy flexible (Org > Region > Branch > Dept), use a "Closure Table" approach in your database. This allows for deep nesting and easy "reporting up" queries.

9. Audit Logging Strategy

A simple log table isn't enough for compliance. You need a **Non-Repudiation** layer.

- **Immutable Logs:** Audit logs should be "Append-Only." No user—not even the Super Admin—should have the DELETE or UPDATE permission on the audit_logs table.
- **Contextual Metadata:** Capture the request_id (to trace the specific API call) and the user_agent (to identify the browser/device).
- **Sensitive Data Masking:** Ensure that the "Old Value" or "New Value" columns in your JSONB never store passwords, OTPs, or full credit card numbers.

10. The "Super Admin" Master Setup

The Super Admin requires a "God-View" dashboard to manage the ecosystem:

- **Tenant Provisioning:** Ability to "Spin up" a new organization, which auto-generates default roles and system masters.
- **Master Data Management:** Centralized control over global constants (Country codes, Currency, Industry types).
- **Usage Analytics:** Monitor active users and storage per organization.

11. Dynamic RBAC (Role-Based Access Control)

- Since role names are flexible, do not hardcode checks like `if (user.role == "Manager")`.
- **Permission Mapping:** Use a **UUID-based Permission System**.
 - *Example:* Define a permission `CAN_VIEW_REPORTS`.
 - The Admin creates a Role (e.g., "Senior Lead") and maps `CAN_VIEW_REPORTS` to it.
 - The code only checks for the UUID/slug of the permission, regardless of what the role is named.
- **Granular Scoping:** Ensure permissions cover three levels: **Read, Write, and Delete**.

12. UI/UX & White-Labeling (Branding)

- **CSS Variables:** Use CSS variables (Custom Properties) for colors.
 - *Example:* `--primary-color: #3498db;`
 - The Super Admin panel updates these values in a theme table, which is loaded at runtime.
- **Asset Storage:** Store logos in a cloud bucket (S3/GCS) with a path structured by `tenant_id`.
- **Dark Mode Toggle:** Use a `prefers-color-scheme` listener by default, but allow a manual override saved in the user profile.

13. Operational "Must-Haves" for SaaS

A. Database Indexing Strategy: A flexible hierarchy can become slow. You must index:

- org_id on every table.
- mobile_number for fast login lookups.
- ancestor_id and descendant_id in the hierarchy table to keep tree-traversal fast.

B. Graceful Degradation (Circuit Breakers)

- If the SMS gateway for OTPs goes down, your entire login flow shouldn't crash. Implement a **Circuit Breaker** pattern that alerts the Super Admin immediately and perhaps falls back to an email OTP if configured.

C. Rate Limiting (Brute Force Protection): Since login is based on Mobile + OTP, implement strict rate limiting:

- Max 3 OTP requests per 10 minutes per IP.
- Max 5 failed attempts before a 15-minute lockout.

D. Soft Deletes

- Never physically delete data from the database. Use a deleted_at timestamp. This allows for "Undo" functionality and preserves the integrity of your Audit Logs.

14. Dynamic Organization Setup Workflow

When a Super Admin creates a new organization, the system should trigger a **Provisioning Script** that:

1. Generates a unique tenant_id.
2. Clones "Default Roles" from a template (e.g., Standard Admin, Standard User).
3. Seeds the org_units table with a root node (The Company Name).
4. Sets the default Branding (Colors/Logo) into the configuration table.

15. Bulk Operations & Data Validation

- Manual entry is the enemy of scale. Implement a robust **Bulk Upload Engine**.
- **The "Staging Table" Pattern**
- Don't insert Excel data directly into your production users table. Use a staging process:
 1. **Upload:** User uploads CSV/Excel.
 2. **Validate:** The system checks for duplicates, formatting (valid mobile numbers), and existence of org_units.
 3. **Error Report:** If 5 out of 100 rows are bad, allow the user to download an "Error Report" showing exactly which rows failed and why.
 4. **Commit:** Only when the data is 100% valid does it move to the users and user_roles tables.

16. Authentication & Identity Management

- **Dual-Phase Onboarding:** For first-time users, the system must perform a "Identity Match" against the pre-loaded Employee Name + Mobile Number before triggering the initial OTP.
- **Device Fingerprinting:** To implement "Common Device" logic, use browser/device fingerprinting (storing a secure, rotating device_id in local storage or cookies).
 - If device_id is missing or unrecognized → Force SMS OTP.
- **Session Management:** Implement JWT (JSON Web Tokens) with short lifetimes and Refresh Tokens to maintain security across web and mobile.

17. Critical Additions for Support & Functionality

- To make the system truly "Enterprise Ready," I recommend adding these three components:

A. Audit Trails (The "Who-Did-What" Log)

- In a flexible system, users will accidentally change role names or move hierarchy nodes. You **must** log every change:
 - **Timestamp | UserID | Action | Old Value | New Value**

B. Global Search & Shortcuts

- As the hierarchy grows, navigation becomes difficult. Implement a "Command Palette" (Ctrl+K) that allows admins to jump directly to a specific employee, role, or report.

C. Impersonation Feature

- For support purposes, allow Super Admins to "Log in as" a specific user (with strict logging) to troubleshoot issues without asking for the user's OTP.

18. Access Enforcement

- In a master-driven SaaS environment, **Access Enforcement** must move away from "If-Else" hardcoding and toward a **Centralized Authorization Middleware**. This layer acts as the final gatekeeper, ensuring that even if a user knows a URL, they cannot see the data or the interface unless their active role possesses the necessary permission UUID.
- The "Permission-First" Middleware Logic
- Multi-Platform Enforcement Strategy
- Mobile Module Enforcement
- Report & Data Scoping

Thank You



MANISH & ANIL



XXXXX XXXXX